

Amazon Connect

Voice Flow Automation

AWS · boto3 · pytest · LocalStack · Chime SDK · Lex V2

A production-ready Python testing framework for
contact centre flows, Lambda functions, and Lex bots

Voice Flow Testing

Lambda Regression

Lex Bot NLU

February 2026

Agenda



1 • What Is This Framework?

Purpose, scope, and key capabilities



2 • Architecture Deep Dive

How Chime SDK drives Amazon Connect calls



3 • Voice Flow Testing

DTMF scripting, CTR validation, CloudWatch



4 • Lambda & LocalStack Testing

Containerised regression without cloud costs



5 • Lex Bot NLU Testing

Multi-turn conversations, intent & slot validation

What Is This Framework?

Automated end-to-end quality assurance for AWS contact centre solutions



Automated Voice Testing

Simulates real PSTN customer calls via AWS Chime SDK SIP Media Application and validates routing outcomes in Amazon Connect.



Configurable Test Cases

All test scenarios are defined in JSON — no code changes required to add new call flows, DTMF paths, or expected queues.



Deep Flow Validation

Verifies contact attributes, CTR records, CloudWatch logs, and DynamoDB state — not just whether the call connected.

3

Testing Modules

Voice · Lambda · Lex

15+

Voice Test Cases

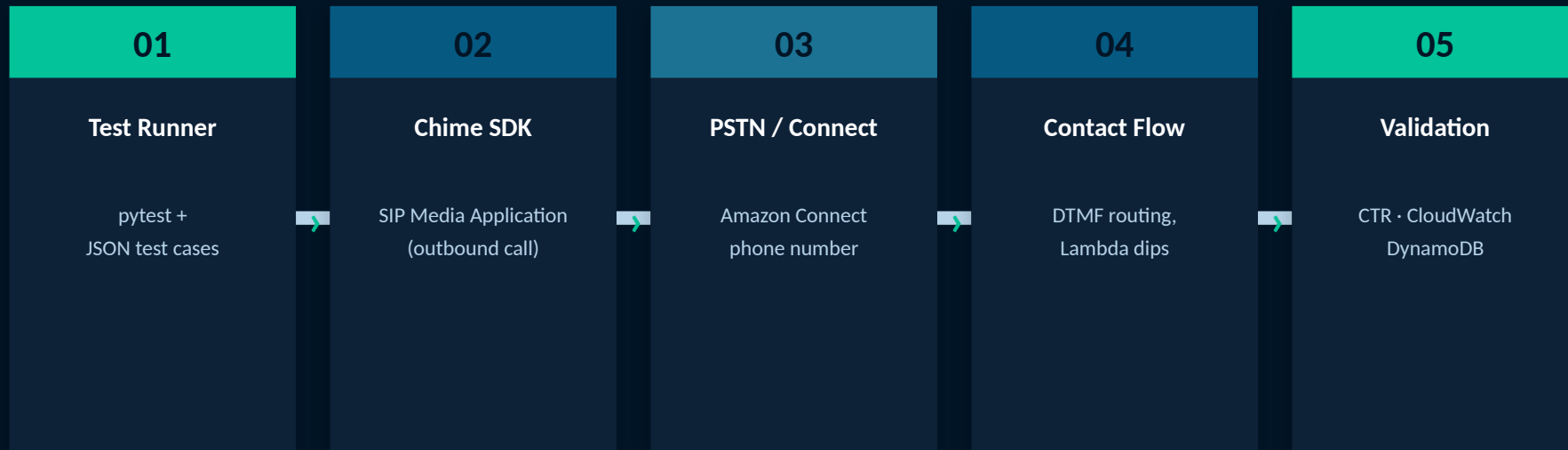
DTMF, multi-turn, error paths

100%

Local Execution

MOCK_AWS mode — no cloud costs

Architecture: How The Framework Works



The test runner reads JSON test cases and uses Chime SDK's `CreateSipMediaApplicationCall` to place a real outbound SIP call.

The Chime Lambda handler executes the call script: TTS prompts, DTMF tones, and wait steps to drive the IVR.

After the call, the framework polls Connect's CTR, CloudWatch Logs, and DynamoDB to validate every expected outcome.

Voice Flow Testing

End-to-end call simulation using AWS Chime SDK SIP Media Application



First-call & returning-caller scenarios — validates greetingPlayed attribute branching



DTMF scripted menus — Press 1 (Queue), Press 2 (Secure Input), Press 3 (Lambda dip)



Multi-turn & reprompt paths — simulates no-input, invalid input, and retry loops



Transfer tracking — verifies sub-flow transfers (SampleSecureInput, SampleLambdaIntegration)



DynamoDB side-effect validation — seeded items cleaned up via pytest finalizer + TTL



CloudWatch Logs — contact flow execution path verified via Logs Insights queries

Sample Test Case

CF-E2E-001 · First Call Happy Path
"Press 1, Join BasicQueue"

Expects:

- greetingPlayed = true
- Queue = BasicQueue
- CTR status = Queued

Poll Timeouts

Queue check	60s	5s poll
CTR record	300s	10s poll
CloudWatch	120s	5s poll
Transcribe	300s	15s poll

Lambda Regression Testing • LocalStack

Full AWS service emulation in Docker — no cloud credentials required



1 • Spin Up LocalStack

Testcontainers spins up LocalStack Docker image. Lambda, S3, DynamoDB emulated locally.



2 • Deploy & Configure

ZIP-packages sample_lambda.py, creates functions, S3 buckets & DynamoDB tables per test case setup block.



3 • Invoke & Validate

Sends trigger_event payload → checks statusCode, response body, DynamoDB items, S3 objects, and FunctionError payloads.



4 • HTML + JSON Report

pytest-html generates a rich visual report; conftest.py emits a structured JSON result for CI artefact upload.

TC-001

S3 → DynamoDB

event trigger

TC-002

DynamoDB streams

error path

TC-003

Async invocation

side-effect check

0

Cloud API calls

in local mode

Lex Bot NLU Testing

Automated intent recognition, slot filling, and multi-turn conversation validation



Single-turn tests

Validates intent, slot values, and response messages for a single user utterance.



Multi-turn conversations

Full dialog flows: ElicitSlot → ConfirmIntent → Fulfilled — tested end-to-end in sequence.



Fallback intent testing

Deliberately sends unrecognised input; validates FallbackIntent is triggered correctly.



Session attribute injection

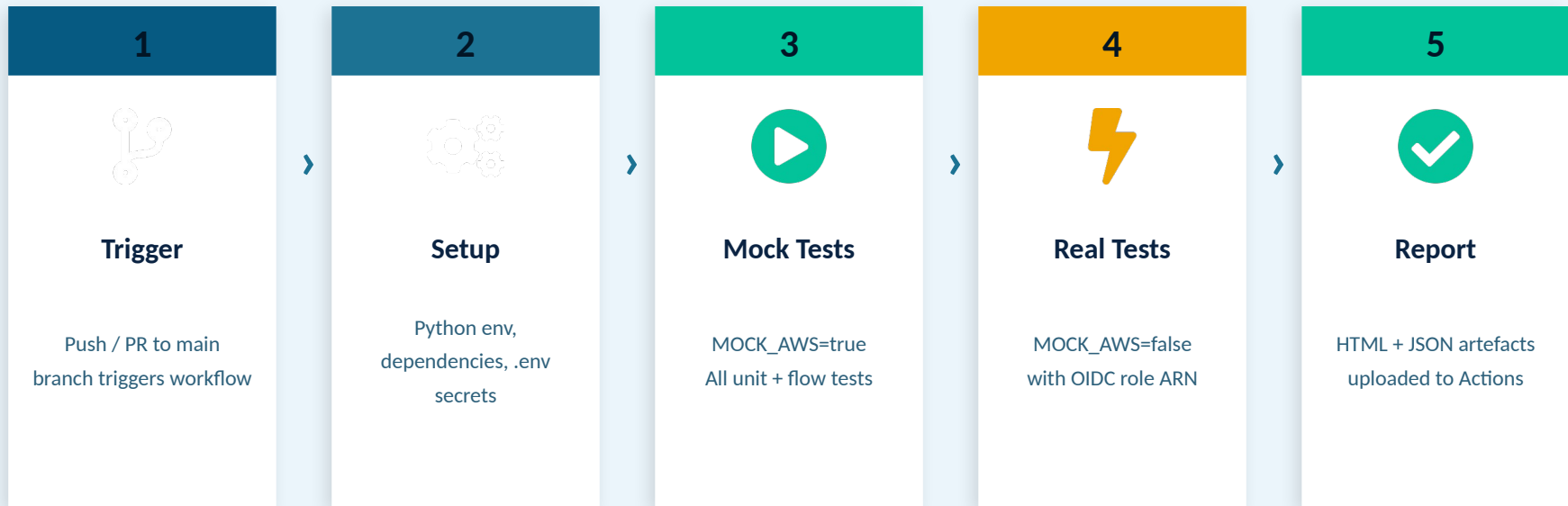
Pre-injects session attributes at test start; validates them at every turn of the conversation.

Dialog States Covered

ElicitSlot	Asking for missing slot
ConfirmIntent	Confirm before fulfilling
Fulfilled	Intent fulfilled
ReadyForFulfillment	Ready for external call
Failed	Dialog failed / aborted
Close	Conversation ended

CI/CD Integration • GitHub Actions

Automated test execution on every push and pull request to main



Required GitHub Secrets

AWS_ROLE_ARN

OIDC trust role ARN for GitHub Actions

AWS_REGION

e.g. us-east-1 or eu-west-2

CONNECT_INSTANCE_ID

Amazon Connect instance ID or ARN

Technology Stack

Tools, libraries, and AWS services used across all three testing modules

Language	Test Runner	AWS SDK	Cloud Emulator	Docker Mgmt
Python 3.x	pytest	boto3 / botocore	LocalStack	Testcontainers
AWS Service	Call Simulation	NLU / Chatbot	Serverless	NoSQL DB
Amazon Connect	AWS Chime SDK	Amazon Lex V2	AWS Lambda	Amazon DynamoDB
Object Storage	Observability	CI/CD	Reporting	Config
Amazon S3	CloudWatch Logs	GitHub Actions	pytest-html	python-dotenv

Test Coverage Overview

What is tested and validated across all three modules

Voice Flows

- ✓ Happy path – first call (DTMF 1)
- ✓ Returning caller skip-greeting
- ✓ DTMF 2 – Secure input sub-flow
- ✓ DTMF 3 – Lambda integration flow
- ✓ DTMF 4 – Note-taking sub-flow
- ✓ DTMF 5 – Chat transfer
- ✓ After-hours closed behaviour
- ✓ Invalid DTMF reprompt
- ✓ No-input timeout reprompt
- ✓ Catastrophic error fallback
- ✓ Callback scheduling
- ✓ Multi-turn auth (DTMF PIN entry)

Lambda Functions

- ✓ S3 PutObject event → DynamoDB write
- ✓ DynamoDB stream event processing
- ✓ Async invocation side-effects
- ✓ FunctionError (error-path cases)
- ✓ Missing env var error handling
- ✓ Response body validation
- ✓ HTTP statusCode assertion
- ✓ S3 object creation validation

Lex Bots

- ✓ Intent recognition accuracy
- ✓ Slot filling & elicitation
- ✓ ConfirmIntent dialog state
- ✓ Multi-turn conversation tracking
- ✓ Fallback / unrecognised utterance
- ✓ Session attribute propagation
- ✓ Active context tracking
- ✓ Response message verification

Getting Started

Clone the repository

```
git clone <repo> && cd amazon_connect_testing
```

Install Python dependencies

```
pip install -r requirements.txt
```

Configure environment variables

```
cp .env.example .env && vim .env
```

Run voice tests in mock mode

```
MOCK_AWS=true pytest voice_testing/
```

Run Lambda tests with LocalStack

```
./lambda_testing/run_lambda_tests.sh
```

Run Lex bot tests

```
pytest lex_testing/ -v --html=report.html
```